# Mapping of Semantic Concepts to Object-Oriented Programming Concepts

Sanyam Asthana

May 2026

## 1 Introduction

I originally wrote this paper back in February, and it had some more stuff in it, and was much more formal, but I have stripped down a lot of stuff from this, for the sake of making it public.

## 2 Prior work

The prior work which comes somewhat close to the idea I propose is DDD (Domain-Driven Development), where programmers define the structures and hierarchies in a way relevant to the problem in hand.

## 3 Methodology

In this section, we discuss how we can draw relations between concepts of formal semantics and theory of object orientation. Throughout the discussion, all the implementations of the semantic concepts will be done in the Java Programming Language, the reason for which will be evident later.

### 3.1 Hyponymy

#### 3.1.1 Definition

Hyponymy is the phenomenon in which a word or an semantic entity contains the meaning of another such entity. For example, "Dog" entails the meaning of "Animal".

#### 3.1.2 Implementation

Hyponymy can be implemented in Java using inheritance. Inheritance is one of the "four pillars" of object orientation wherein one class inherits the properties of its parent class.

```
public class Animal {
    ...
}

public class Dog extends Animal {
    ...
}
```

Here, "Dog" inherits the attributes of the "Animal" class. We use the 'extends' keyword here to inherit the Animal class.

In normal language, when we make a statement about a dog, it also applies to the statement about an "Animal". Similarly, when an object of the 'Dog' class is created here, all methods and attributes of the 'Animal' class apply to the object as well.

There is one change I would like to make here, which is explained in section 3.2.

## 3.2 Prototype theory

### 3.2.1 Definition

The prototype theory of Rosch says that humans have a fuzzy, abstract idea, or a prototype of a concept. For example, when "furniture" is mentioned, it evokes certain ideas of the following object in the brain.

### 3.2.2 Implementation

These abstract, incomplete ideas are best modeled using Abstract classes in Java. The previous code is modified to redefine the 'Animal' class as an abstract class. After all, a standalone "Animal" can not be found in the world, but only certain classes of animals.

```
public abstract class Animal {
    ...
}

public class Dog extends Animal {
    ...
}
```

Dog is not an abstract class since there are instances of Dogs which can be found, but no particular instance of "Animal".

## 3.3 Semantic agreement

### 3.3.1 Definition

Semantic agreement refers to the agreement between the noun and the verb related to the semantic functions of the noun, and validity of the verb regarding those functions. For example, the verb "fly" does not agree with the noun "dog", as a dog is incapable of flying.

### 3.3.2 Implementation

Classes can be given these "abilities" in the form of interfaces. Interfaces in Java are an abstract structure which defines the attributes and methods a class must implement.

```
public interface Walkable {
    walk()
    tripOver()
}

public abstract class Animal {
    ...
}

public class Dog extends Animal implements Walkable {
    ...
}
```

This is where the semantic rules of language start to come into play. In this example, an entity which cannot walk (does not implement the Walkable interface) cannot trip over.

Interfaces can also be used to assign certain characteristics to classes.

```
public interface Walkable {
    walk()
    tripOver()
}

public interface Domestic {}

public abstract class Animal {
    ...
}

public class Dog extends Animal implements Walkable, Domestic {
    ...
}

public void pet(Domestic domesticable) {
    ...
}
```

Notice how the interface 'Domestic' is empty. The interface serves no purpose other than being a characteristic which other methods, like ‘pet()‘ can utilize.

## 3.4 Meronymy

### 3.4.1 Definition

Meronymy is the phenomenon in which an entity has other entities as a part of it.

### 3.4.2 Implementation

Meronymy can be implemented in Java using objects of other classes as attributes of an object.

```
public class Tail {
    ...
}

public class Nose {
    ...
}

public class Dog extends Animal implements Walkable, Domestic {
    Tail dogTail;
    Nose dogNose;
}
```

Here, the class Dog has 'dogTail' and 'dogNose' as its components, similar to how a dog "comprises" of (but is not limited to) a tail and a nose.

## 3.5 Synonymy

### 3.5.1 Definition

Synonymy is the phenomenon in which two different words reflect the same (or almost same) ideas. Hence, in general, one can be substituted for the other (though there are exceptions).

### 3.5.2 Implementation

Interestingly, Java does not have an inbuilt framework to model synonymy. We can assign a variable the reference to a class or an object, but that reference would be a pointer, and not a true alias.

The closest we can get to a true alias is in the case of C/C++ where the 'typedef' keyword is used, or in Haskell, where the 'type' keyword is used. However, these languages fail to be capable of modeling the other semantic relations.

## 3.6 Signifier (Words)

### 3.6.1 Definition

I will make the clear distinction that "words" here represent the "signifier" part, and not the "signified" part. This means that the "words" are taken as how they appear at face value, and not what they actually signify. The reason for this will be evident in section 3.7.

### 3.6.2 Implementation

Words can be implemented by classes in Java.

```
public class Dog {
    ...
}
```

In this example, "Dog" is a word.

## 3.7 Homonymy/Polysemy

### 3.7.1 Definition

**Homonymy:** The phenomenon in which one word has two separate unrelated meanings. Eg. "bat" meaning the flying animal, and the instrument used for playing baseball.

**Polysemy:** The phenomenon in which one word has two seemingly unrelated meanings, but the origins of the two meanings stem from the same root meaning. Eg. "bank" referring to the river bank and bank as a financial institution.

### 3.7.2 Implementation

Both the categories are merged together in one entry because of two reasons:

- Java does not have a complete framework to distinguish between unrelated and somewhat related interfaces.
- The distinction between homonymy and polysemy is tough to make even in the field of formal linguistics.

They can be implemented using multiple interface implementation in Java.

```
public class Book implements Readable, Doable {
    ...
}
```

## 3.8 Antonymy

### 3.8.1 Definition

Antonymy is the phenomenon in which two words of the same category have contrasting meanings. Antonymy can be of different types (complementary, gradable and converse).

### 3.8.2 Implementation

Antonymy does not have a clear implementation in Java, or any other language for that matter. The closest a language comes to modeling (gradable) antonymy is Haskell, through its 'range' system.

Because type and class hierarchies model classification, and antonymy is about opposition, there might fundamentally not be a unified framework to model this in any programming language.

# 4 Discussion

## 4.1 Limitations

I have now mapped the semantic relations to features in Java. However, there are certain limitations to this model, notably, Synonymy and Antonymy. Synonymy is not a fundamental issue, as there are languages other than Java which implement it perfectly fine, but antonymy could be a more fundamental issue, where there may not be a unified framework to capture all the forms of antonymy.

## 4.2 Other languages

I would also like to point out the reason to use Java in this study. Java has quite a verbose and strict type system, and it also features a lot of constructs for the hierarchies.

Something like C++ would not have worked because of the lack of abstract classes and interfaces (abstract classes can be implemented using virtual methods, though).

Kotlin would not have worked, since it is not strictly object oriented, as functions can live outside classes, and it does not have a type system as strict as Java's (type inference in Kotlin).

I define the terms **Semantically Complete** and **Semantically Competent**. A programming language is said to be Semantically Complete if it can accurately model all of the semantic relations. Unfortunately, I was unable to find a true semantically complete language. However, some languages come closer to this completeness than the others, making them more semantically competent than the others, like Java.